Hamzstlab Mathematics and Hamzstplot

DS Glanzsche[1]

Freya

Hamzst

2025 Edition

[1]A thank you or further information

Contents

Preface

For my Wife Freya, and our daughters Solreya, Mithra, Catenary, Iyzumrae and Zefir.

For Lucrif and Znane too along with all the 8 Queens.

For Albert Silverberg, Camus and Miklotov who left TREVOCALL to guard me.

To Nature(Kala, Kathmandu, Big Tree, Sentinel, Aokigahara, Hoia Baciu, Jacob's Well, Mt Logan, etc) and my family Berlin: I have served, I will be of service.

To my dogs who always accompany me working in Valhalla Projection, go to Puncak Bintang or Kathmandu: Sine Bam Bam, Kecil, Browni Bruncit, Sweden Sexy, Cambridge Klutukk, Milan keng-keng, Piano Bludut and more will be adopted. To my cat who guard the home while I'm away with my dogs: London.

The one who moves a mountain begins by carrying away small stones - Confucius

A book to explained how we create a C++ library from forking / branching out ImGUI 1.92.0 WIP, Implot, Implot 3D, merge them into one big pile that we call Hamzstlab Mathematics so it can be easily compiled / build so we can focus more on the Mathematics side / learning.

We also write about Hamzstplot that is forking / branching out from Matplot++, it is a C++ plotting library with backend gnuplot, so you can have more creativity in plotting since the syntax will be all C++. It is not as interactive as Hamzstlab Mathematics but it has more options of plotting that you can play with.

a little about GlanzFreya:
Freya the Goddess is (DS Glanzsche') Goddess wife. We get married on Puncak Bintang on November 5th, 2020 after we go back from Waghete, Papua. My wife birthday (Freya the Goddess) is on August 1st. After not working with human anymore since 2019, I (DS Glanzsche) work in a forest, shaping a forest into a natural wonder / like a canvas for painting, and also clean up trashes that are thrown in the forest. Thus after 6 years working with Nature I have found what can makes me waltz to work, it is going to the forest in the morning then go back home again and study / learn science, engineering, computer programming.

a little about Hamzst:
Hamzst is a nickname for Alice from Persona game series made by Atlus, sometimes she can also be similar with Alice from Alice in Wonderland (1951 movie). When she said "But in my world, the books will be nothing but picture," it is true for Hamzstplot and Hamzstlab Mathematics.

Figure 1: *Freya, thank you for everything, I am glad I marry you and I could never have done it without you.*

Figure 2: *I paint her 3 days before Christmas in 2021.*

Chapter 1

Introduction Story

On top of the mountain a heart shaped stone waiting for You, my eyes can't see, my body can't touch, but my heart knows it's You. Forever only You. - ***Glanz to Freya***

This book is written on May 29th, 2025, while we also have another project to make SymIntegration C++ library, we are working on this when we saw Implot, Implot 3D, and Matplot++. We want to clone them and modify them, or probably add some features to plot, simulate and more in the future, it is called branching out / forking. Then rename it as Hamzstlab Mathematics for combining Imgui+Implot+Implot3D+Imnodes, as a C++ library / software where you can create interactive graph or simulation for mathematics problems. It will be very useful for students and researchers. While Hamzstplot is branching out from Matplot++ to create graph and plot as well but it is not interactive as Hamzstlab Mathematics but the options for building the plot are enormous. We can choose which one to use depends on our need.

To avoid confusion in the future: Whenever there is a written of subject "I" it means DS Glanzsche as the I subject.

Years ago, around December 2019 when I (DS Glanzsche) was hiking / jogging up to Puncak Bintang, I pray at dawn, I remember I used to arrive there at 4:30 AM , departing around 3:00 AM from home, what I pray is " I pray and hope one day I can create a software like MATLAB," well I am starting this and remember that prayer again, it is becoming true, when I was installing MATLAB back then in 2011 to 2014 for college purpose, they give MATLAB course to students and even pay the license for it, I think it is great, but then realize, when I was reading a book like "Elementary Linear Algebra" or ""Elementary Differential Equation" or "Calculus" none of them have the MATLAB code to show how the plot is made or the computation is being computed, even if today there is a book called "Advanced Engineering Mathematics with MATLAB" we still need the code for the basic that undergraduate needs, since the knowledge for other science and engineering department will rely heavily on those Calculus, Linear Algebra, and Differential Equations first.

Not to mention, that there are tons of menu / on the menu bar on MATLAB, that the user won't even need to use for the rest of his/her life, the size of MATLAB today is over 20 GB, and for the installation it takes hours, now I am comparing with Hamzstlab Mathematics, if you are an Electrical Engineer, and we already create the basic code and API to create any electric circuit with basic NPN' or PNP' BJT transistor or MOSFET n-channel or p-channel along with any number of resistors, inductor, and capacitor and will have the computed voltage and current, and you

probably won't even need to download Hamzstlab Mathematics for even 1 GB, isn't it going to save your day and your time?

This is the real life example where we try to plot a solution to a simple first order ordinary differential equation and the Euler' method approximation, the size of the C++ file and the Makefile only consumes 3.1 KB, and if you build the example the C++ codes it will create this object files that will consumes 27.3 MB of size of your storage, which can be cleaned after you are finish. Hamzstlab Mathematics will only consume your storage of 50.3 MB, and the rest will depends on the project that you will be working on.

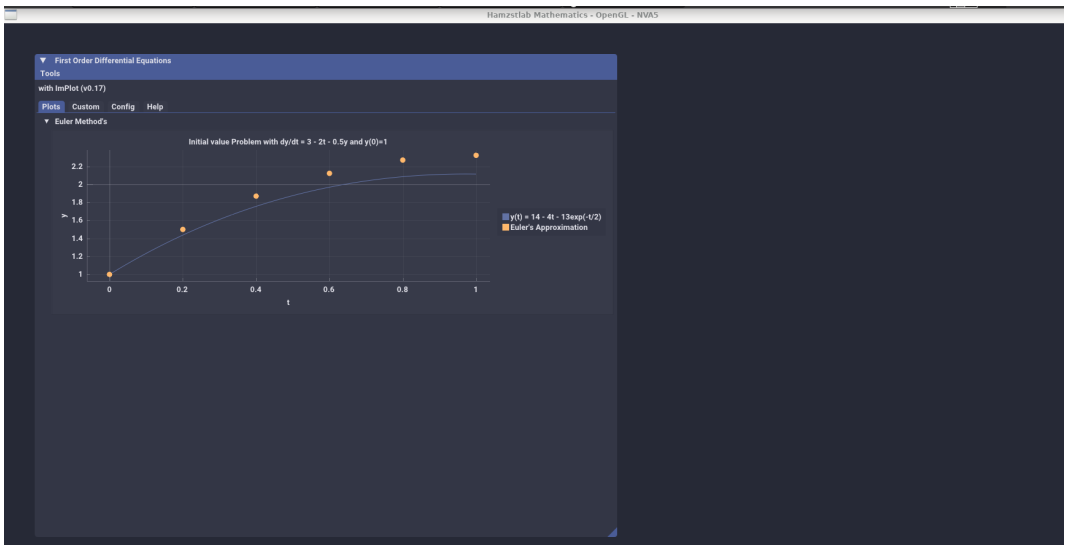

Figure 1.1: *The interactive plot of Euler' method approximation and the solution plot for* $\frac{dy}{dt} = 3 - 2t - \frac{1}{2}y$ *with Hamzstlab Mathematics.*

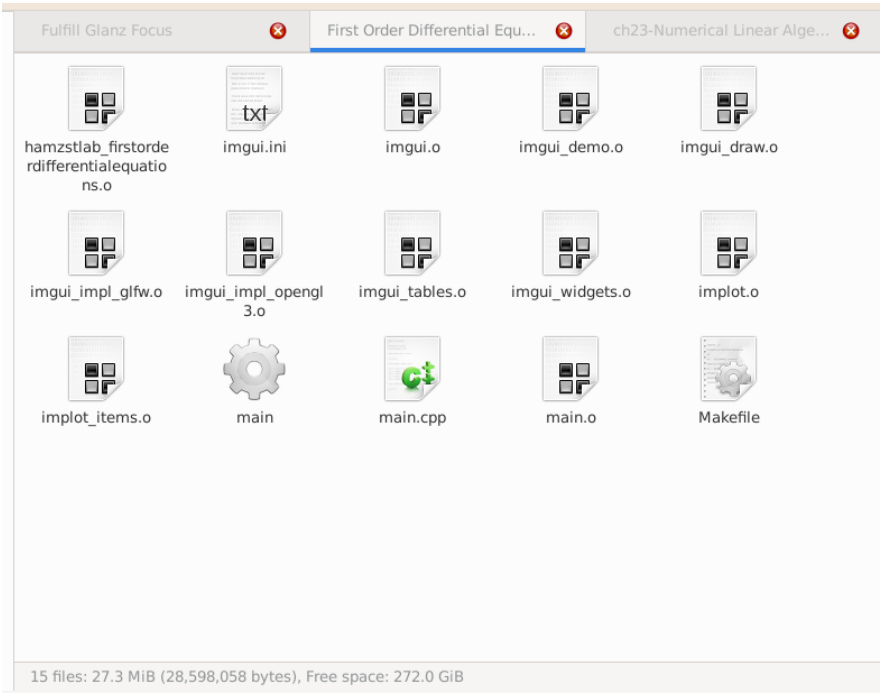

Figure 1.2: *The size of a built example with all the object files.*

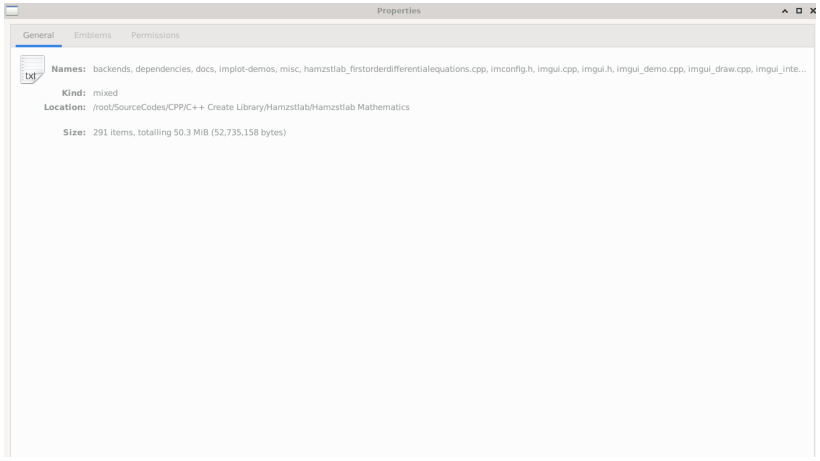

Figure 1.3: *The size of Hamzstlab Mathematics source code as of June 4th, 2025.*

On May 26th, 2025 while in St Peter Cathedral in Bandung I was reading a book [4], then at the first chapter it stated like this:
If all you are interested in is a quick calculation, then Python along with its ecosystem is likely going to be your one-stop shop. As your work becomes more computationally challenging, you may need to switch to a compiled language; most work on supercomputers is carried out using languages like Fortran or C++ or sometimes even C.

Basically, what Imgui combined with Implot and Implot 3D now able to do, show a lot of

interactive visualizations in one window, is a big gap between C++ and Python, like the sky and the earth. I (DS Glanzsche) as one of the author, starts learning C++ rigorously since October 2023, still very infant stage, then able to write DianFreya Math Physics Simulator and already finish the whole Elementary Linear Algebra book along with all the C++ codes for plotting and simulation. Previously, I was using Python and JULIA to do the computation while writing the book Lasthrim Projection (focusing on Calculus and basic Mathematics field that are more theoretical like Discrete Mathematics, Elementary Differential Equations, Partial Differential Equations). Now I have learned Python, JULIA, C++, Fortran, and C. I know writing a code with C++ to achieve the same result will take longer and steeper learning curve, but it is worth the time spent.

The Operating System and Softwares in Use:

1. GFreya OS version 1.8 (based on LFS and BLFS version 11.0 System V books)

2. GCC version 11.2.0 (to compile C++ codes)

3. Hamzstplot version 1.5

4. Hamzstlab Mathematics version 1.51

We will explain about Hamzstplot and Hamzstlab Mathematics in the next two chapters, then the rest of the chapters will be talking about their application (in creating the plot, interactive plot, or simulation with either Hamzstplot or Hamzstlab Mathematics).

We are really appreciate those who spend time to write constructing critics, not hate words without reason that will only add more good luck and karma to me, write their opinions on what should be added in SymIntegration or if there is an incorrect C++ codes or algorithm to compute the integrals. We cannot provide the readers who send nice feedbacks with anything now, but if we have money, when someone send / email me a very good feedbacks then we will send you either cookies, doughnuts or parcel of foods. Feedbacks can be sent to **dsglanzsche@gmail.com**.

Chapter 2

Hamzstplot

"The best and most beautiful things in the world cannot be seen or even touched - they must be felt with the heart" - ***Helen Keller***

Hamzstplot was born on a Full Moon day on May 12th, 2025. We are branching out / forking from Matplot++ and we want to develop it further, changing the name first and will add several features in the future to help us in learning Mathematics, or other branch of science, e.g. to create direction fields. The basic syntax of C++ will really help for a very long term future prospects and more complex problems in science to be computed, analyzed and plot.

I. Build and Install Hamzstplot in GFreya OS

We are using GFreya OS as it is a custom Linux-based Operating System, your distro your rules. The commands here assuming that you / the reader are using Linux-based OS,they are familiar and easy to follow for other Linux-based OS users.

To download Hamzstplot, open terminal / xterm then type:
git clone https://github.com/glanzkaiser/Hamzstplot.git
Enter the directory then type:

```
mkdir build
cd build
cmake —DBUILD_SHARED_LIBS=ON ..
make
make install
```

Code 1: *Create Hamzstplot dynamic library*

It will create the dynamic library **libhamzstplot.so**, we are using CMake, and it is said that dynamic library saves more RAM memory than static library.

The prefix at default is **/usr/local** so the installed dynamic library will go to **/usr/local/lib**.

Then open terminal and from the current working directory / this repository main directory:

```
cd /source/3rd_party/cimg/
cp —r CImg.h* /usr/local/include

cd /

cd ../Hamzstplot/source/hamzstplot/
cp —r * /usr/local/include

cd ../Hamzstplot/source/3rd_party/nodesoup/include/
cp —r nodesoup.hpp /usr/local/include

cd /
cd ../Hamzstplot/source/3rd_party/nodesoup/src/
bash dynamiclibrary.sh
cp —r libnodesoup.so /usr/local/lib
```

Code 2: *Move include files and 3rd party library*

When we want to create the shared library it will look for this header files in the default path where the include files are usually located. In Linux OS **usr/include** is the basic / default path, another default path is **usr/local/include**.

All files, examples codes and source codes used in this books can be found in this repository:
https://github.com/glanzkaiser/Hamzstplot

II. Example of Plotting with Hamzstplot in GFreya OS

We will create a 3D plot of a helix, the source code is this:

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

int main() {
	using namespace hamzstplot;

	auto xt = [](double t) { return sin(t); };
	auto yt = [](double t) { return cos(t); };
	auto zt = [](double t) { return t; };
	fplot3(xt, yt, zt);

	show();
	return 0;
}
```

Code 3: *Hamzstplot/Examples with Makefile/Plot 3D Helix/main.cpp*

Open terminal and from the current working directory **Hamzstplot/Examples with Makefile/-Plot 3D Helix/**, then type:

```
make
./main
```

Code 4: *Build and run the executable*

or you can also type in the current working directory (a manual way to compile):
g++ main.cpp -o main -lhamzstplot
./main

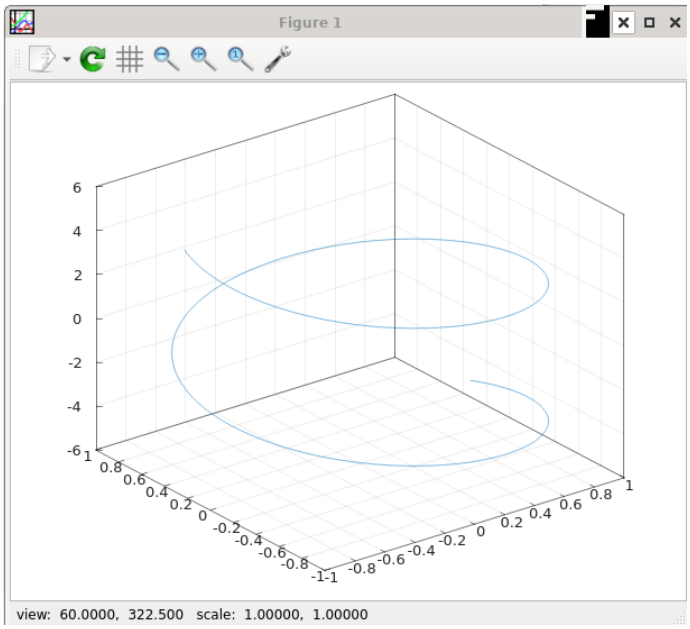

Figure 2.1: *The plot here is using the gnuplot as the backend, that is why the GUI window should look very familiar.*

Chapter 3

Hamzstlab Mathematics

"In this world perhaps, but in my world, the books will be full of pictures" - ***Alice (Alice in Wonderland - 1951)***

Why would we need Hamzstlab Mathematics if we already have Hamzstplot? For the extra interactive feature, better simulation, even if the backend of gnuplot or OpenGL from Hamzstplot is already good enough, the features in Hamzstlab Mathematics that is coming from ImGUI, Implot, Implot3D, Imnodes can reach to almost all branches of science and engineering, help them to design and make prototype. Box2D is using ImGUI and it helps me personally to create some basic Physics simulation from an undergraduate book "Fundamental of Physics."

You can design any kind of electric circuit here with all the computation necessary too, you can simulate how a mechanical system works, how leverage works, how a human organ works, how a nuclear fission works, and you can even do gene sequence simulation better here, they are all in my head now, but one day we will add the source code and API so people can really create what I am talking now.

In the end, there is no perfect software, one software or one C++ library complement the other. That is life. Just like marriage, since nobody is perfect, just be honest, no lie no cheating, accept your partner for whoever she is, with no extramarrital affair / lie / cheating, then you both will be happy and reach your goal together, with a bonus become power couple too.

I. Build and Install Hamzstlab Mathematics in GFreya OS

We are using GFreya OS as it is a custom Linux-based Operating System, your distro your rules. The dependencies are:

1. ImGUI version 1.92.0 WIP
2. Implot version 0.17
3. Implot-3D version 0.3 WIP
4. Imnodes

There is no need to download all of the above one by one anymore, because they are already blended into one in Hamzstlab Mathematics.

To download Hamzstlab Mathematics, open terminal / xterm then type:
git clone https://github.com/glanzkaiser/Hamzstlab-Mathematics.git

Enter the directory then type:

```
cd dependencies

cp —r cxxopts.hpp /usr/include
```

Code 5: *Move an include file / header*

For the **implot-demos** we need a few compiling first, from the main directory open the terminal then type:

```
cd implot—demos

mkdir build
cd build
cmake ..
make

cp —r libimnodes.a libimplot.a libapp.a libimgui.a ../../dependencies/
```

Code 6: *Create implot-demos related static library*

The steps above are used to create the static library to run some demos from **implot-demos** that are more complex than the normal implot demo.

Now, you are set to go, in the **Hamzstlab-Mathematics/examples/** directory there are tons of examples that you can just build with Makefile, easy.

For example, from the main directory **Hamzstlab-Mathematics/** open terminal

```
cd examples

cd 3D Test
```

```
make
./main
```

Code 7: *Create dynamic library*

This will run the 3D test. You can check each examples source code and learn how to modify according to your needs.

All files, examples codes and source codes used in this books can be found in this repository: **https://github.com/glanzkaiser/Hamzstlab-Mathematics**

II. Example of Creating an Interactive Plot with Hamzstlab-Mathematics in GFreya OS

First you have to already clone or download the repository, then open terminal at the main working directory **Hamzstlab-Mathematics** and type:

```
cd examples

cd First Order Differential Equation

make
./main
```

Code 8: *Create dynamic library*

This will be the result:

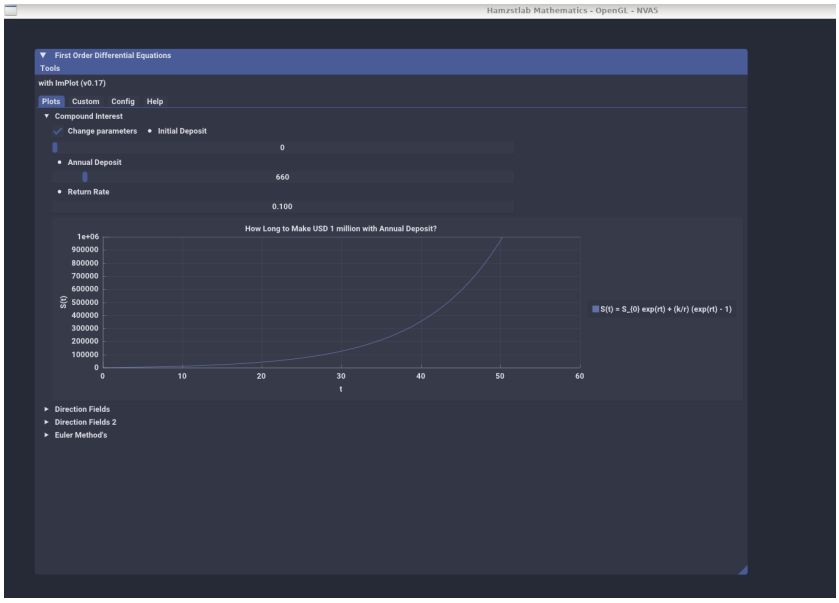

Figure 3.1: *The interactive plot of $S(t)$ with Hamzstlab-Mathematics you can change the parameters and the plot will adjust with the parameters that you just input.*

Now, we are going to explain the code on how to make this example works.

```
#include "App.h"

struct ImPlotDemo : App {
	using App::App;
	void Update() override {
		ImPlot::ShowFirstOrderDEWindow();
	}
};

int main(int argc, char const *argv[])
{
	ImPlotDemo app("Hamzstlab Mathematics",1920,1080,argc,argv);
	app.Run();

	return 0;
}
```

Code 9: *Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp*

This **main.cpp** is pointing toward **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**, so if you want to modify the example, you just need to edit **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**.

The **ImPlot::ShowFirstOrderDEWindow();** needs to be an **IMPLOT_API**, and this can be done in **Hamzstlab-Mathematics/implot.h**, so you need to add a line like this there

```
...

IMPLOT_API void ShowFirstOrderDEWindow(bool* p_open = nullptr);

...
```

Code 10: *Hamzstlab-Mathematics/implot.h*

For example, from hundred lines of codes you can check the code here to create a plot of Compound Interest, if you scroll down again you will know that we are showing this **Demo_CompoundInterest()** under a new tab that can be opened by the user.

There might be hundreds to thousands of lines of codes, but if you are rigorous and persistence you will get used to it and able to create something from your imagination.

```
...
...

void Demo_CompoundInterest() {
	static float xs1[1001], ys1[1001];
	static int S0 = 0;
	static float r = 0.08;
```

```
            static int k = 2000;
            for (int i = 0; i < 1001; ++i) {
                  xs1[i] = i;
                  ys1[i] = S0 * exp(r*i) + (k/r)*(exp(r*i) - 1) ;
            }

            static bool range = false;
            ImGui::Checkbox("Change parameters", &range);

            if (range) {
                  ImGui::SameLine();
                  ImGui::SetNextItemWidth(200);
                  ImGui::BulletText("Initial Deposit");
                  ImGui::SliderInt("##Initial Deposit", &S0, 0, 10000);
                  ImGui::SetNextItemWidth(200);
                  ImGui::BulletText("Annual Deposit");
                  ImGui::SliderInt("##Annual Deposit", &k, 1, 10000);
                  ImGui::SetNextItemWidth(200);
                  ImGui::BulletText("Return Rate");
                  ImGui::DragFloat("##Return rate", &r, 0.01f, 0.2f);
                  ImGui::SetNextItemWidth(200);
            }
            if (ImPlot::BeginPlot("How Long to Make USD 1 million with Annual
                Deposit?")) {
                  ImPlot::SetupAxes("t","S(t)");
                  ImPlot::SetupAxesLimits(0, 60, 0, 1000000);
                  ImPlot::SetupLegend(ImPlotLocation_East,
                      ImPlotLegendFlags_Outside);

                  ImPlot::PlotLine("S(t) = S_{0} exp(rt) + (k/r) (exp(rt) - 1)
                      ", xs1, ys1, 1001);
                  ImPlot::EndPlot();
            }
      }
      ...
      ...

      void ShowFirstOrderDEWindow(bool* p_open) {
      ...

      if (ImGui::BeginTabBar("ImPlotDemoTabs")) {
      if (ImGui::BeginTabItem("Plots")) {
            DemoHeader("Compound Interest", Demo_CompoundInterest);

      ...
      ...
      void ImPlot::ShowFirstOrderDEWindow(bool* p_open) {}
```

```
#endif
```

Code 11: *Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp*

Last, but not least, for the Makefile **Hamzstlab-Mathematics/examples/First Order Differential Equation/Makefile**, we need to add the SOURCES to be compiled that is pointing toward **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**

```
EXE = main
IMGUI_DIR = ../..
SOURCES = main.cpp
SOURCES += $(IMGUI_DIR)/imgui.cpp $(IMGUI_DIR)/imgui_demo.cpp $(IMGUI_DIR)/
    imgui_draw.cpp $(IMGUI_DIR)/imgui_tables.cpp $(IMGUI_DIR)/imgui_widgets
    .cpp
SOURCES += $(IMGUI_DIR)/backends/imgui_impl_glfw.cpp $(IMGUI_DIR)/backends/
    imgui_impl_opengl3.cpp
SOURCES += $(IMGUI_DIR)/implot.cpp $(IMGUI_DIR)/implot_items.cpp
SOURCES += $(IMGUI_DIR)/hamzstlab_firstorderdifferentialequations.cpp

OBJS = $(addsuffix .o, $(basename $(notdir $(SOURCES))))
UNAME_S := $(shell uname -s)
LINUX_GL_LIBS = -lGL -lglfw

CXXFLAGS = -std=c++11 -I$(IMGUI_DIR) -I$(IMGUI_DIR)/backends
CXXFLAGS += -g -Wall -Wformat
LIBS = ../../dependencies/glad.c -L../../dependencies/ -lapp -limgui -
    limnodes -limplot

...
...
```

Code 12: *Hamzstlab-Mathematics/examples/First Order Differential Equation/Makefile*

Chapter 4

Differential Equations

"Everyone's favorite fighting fairy godmother is still kicking, taking out Ether Strike hits for her godfather Odin. The heretofore unmatched fury of her scowl derives from her lack of lines in the main story" - ***Freya (Valkyrie Profile: Covenant of the Plume)***

Dıfferential equations are equations that contain derivatives. It is used to understand and to investigate problems involving the motion of fluids, the flow of current in electric circuits, the dissipation of heat in solid objects, the propagation and detection of seismic waves [1]. For example, the Newton's second law can be written as a differential equation

$$F = ma \tag{4.1}$$

$$F = m\frac{dv}{dt} = mv' \tag{4.2}$$

We think of v as a function of t, or we can also write it as $v(t)$.

I. Direction Fields

[H*] Direction fields are valuable tools in studying the solutions of differential equations of the form

$$\frac{dy}{dt} = f(t, y) \tag{4.3}$$

where f is a given function of two variables t and y, sometimes referred to as the rate function. A direction field can be constructed by evaluating f at each point of a rectangular grid. At each point of the grid, a short line segment is drawn whose slope is the value of f at that point. Thus each line segment is tangent to the graph of the solution passing through that point.

[H*] A direction field drawn on a fairly fine grid gives good picture of the overall behavior of solutions of a differential equation.

The construction of a direction field is often a useful step in the investigation of a differential equation.

[H*] **Example:**
Consider a population of field mice who inhabit a certain rural area. If we denote time by t and the mouse population by $p(t)$ with the time is measured in months, then the differential

that represents the population growth of field mice that is also preyed on by owls can be expressed by the equation

$$\frac{dp}{dt} = 0.5p - 450$$

Observe that the predation term is measured in months as -450, meaning that each month there is about 450 mouse being preyed by owls every month.

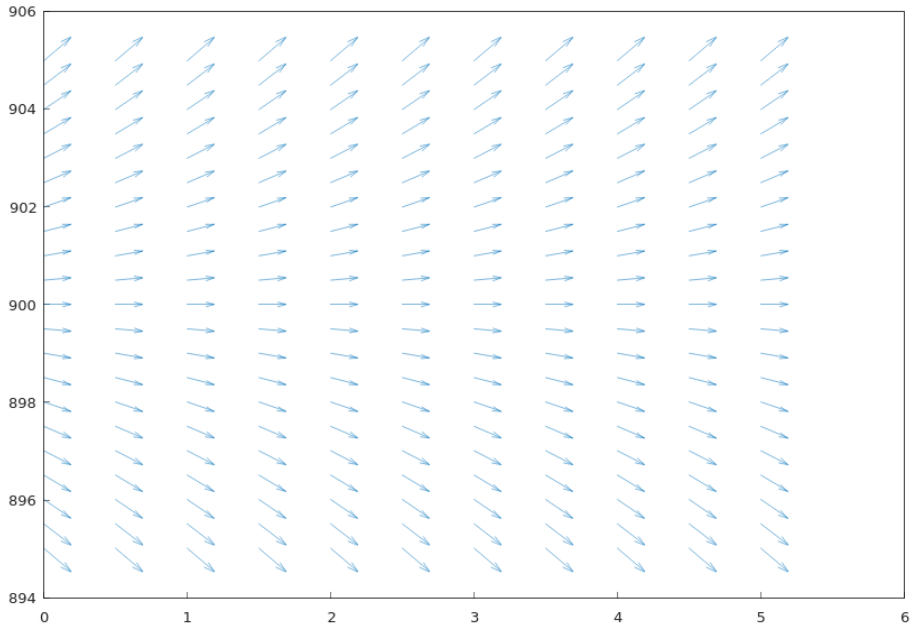

Figure 4.1: *The direction field for $\frac{dp}{dt} = 0.5p - 450$, the x-axis represents the variable t as the time and the y-axis represents variable $p(t)$ as the population growth of field mice, we can see that if the rate of the reproduction for the field mice is greater than the rate of the mice being preyed on by owls then the population growth will keep increasing, and if the rate of the reproduction for the fiel mice is smaller than the rate of the mice being preyed on by owls then the population of field mice will keep decreasing.*

The code to create the direction field above with Hamzstplot:

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;


int main() {
       using namespace hamzstplot;
       double dx, dy, magnitude;
       auto [x, y] = meshgrid(iota(0, 0.5, 5), iota(895, 0.5,
           905));
```

```
                vector_2d u = transform(x, y, [](double x, double y) {
                    return 1 ; }); // dx
                vector_2d v = transform(x, y, [](double x, double y) {
                    return 0.5*y - 450 ; }); // dy
                quiver(x, y, u, v);

                show();
                return 0;
            }
```

Code 13: *Hamzstplot/Examples with Makefile/Plot Direction Fields Growth and Predation Rate/main.cpp*

For sufficiently large values of p it can be seen that $\frac{dp}{dt}$ is positive, so that solutions increase. On the other hand, if p is small, then $\frac{dp}{dt}$ is negative and solutions decrease.

The critical value of p that separates solutions that increase from those that decrease is the value of p for which $\frac{dp}{dt}$ is zero.

By setting $\frac{dp}{dt}$ equals zero and then solving for p, we find the equilibrium solution $p(t) = 900$ for which the growth term and the predation term are exactly balanced.

II. Solving a Simple Differential Equation

[H*] Consider we have this differential equation

$$\frac{dp}{dt} = 0.5p - 450$$

If we want to obtain the solution $p(t)$ we will have to perform some algebraic trick or manipulation then integrating them with respect to the correct variable.

$$\begin{aligned}
\frac{dp}{dt} &= 0.5p - 450 \\
&= \frac{p - 900}{2} \\
\frac{dp/dt}{p - 900} &= \frac{1}{2} \\
\frac{dp}{p - 900} &= \frac{dt}{2} \\
\int \frac{dp}{p - 900} &= \int \frac{dt}{2} \\
\ln|p - 900| &= \frac{t}{2} + C \\
|p - 900| &= e^{\frac{t}{2} + C} \\
|p - 900| &= e^{\frac{t}{2}} e^{C} \\
p - 900 &= \pm e^{\frac{t}{2}} e^{C} \\
p &= 900 + ce^{\frac{t}{2}}
\end{aligned}$$

where $c = \pm e^C$ is also an arbitrary (nonzero) constant. Note that the constant function $p = 900$ is also a solution where $c = 0$.

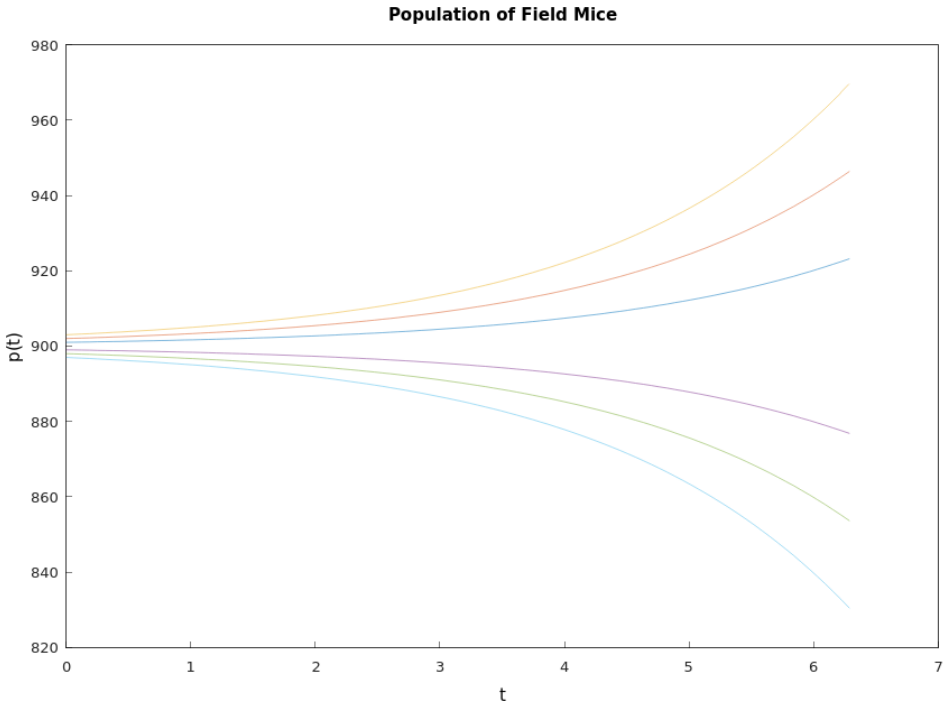

Figure 4.2: *The graphs shows the solutions for* $\frac{dp}{dt} = 0.5p - 450$ *with several valus of c.*

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;

int main() {
       using namespace hamzstplot;

       std::vector<double> t = linspace(0, 2 * pi);
       std::vector<double> y1 = transform(t, [](auto t) {
           return 900 + 1*exp(0.5*t); });
       std::vector<double> y2 = transform(t, [](auto t) {
           return 900 + 2*exp(0.5*t); });
       std::vector<double> y3 = transform(t, [](auto t) {
           return 900 + 3*exp(0.5*t); });
       std::vector<double> y4 = transform(t, [](auto t) {
           return 900 + -1*exp(0.5*t); });
       std::vector<double> y5 = transform(t, [](auto t) {
           return 900 + -2*exp(0.5*t); });
       std::vector<double> y6 = transform(t, [](auto t) {
           return 900 + -3*exp(0.5*t); });
```

```
            plot(t, y1, t, y2, t, y3, t, y4, "—", t, y5,"—", t,y6,
                "—");
            title("Population of Field Mice");
            xlabel("t");
            ylabel("p(t)");
            show();
            return 0;
        }
```

Code 14: *Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Solutions Curves/main.cpp*

III. Classification of Differential Equations

[H*] One important classification is based on whether the unknown function depends on a single independent variable or on several independent variables.

If a function depends on a single independent variable then it is said to be an ordinary differential equation, this is the example:

$$L\frac{d^2Q(t)}{dt^2} + R\frac{dQ(t)}{dt} + \frac{1}{C}Q(t) = E(t) \tag{4.4}$$

with $Q(t)$ represents the charge on a capacitor in a circuit with capacitance C, resistance R, and inductance L.

If a function depends on mroe than one independent variable then it is called a partial differential equation, this is the example:

$$\alpha^2\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t} \tag{4.5}$$

it is called the heat conduction equation with $alpha$ as the physical constant. Another example is:

$$\alpha^2\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial^2 u(x,t)}{\partial t^2} \tag{4.6}$$

it is called the wave equation with α as the physical constant.

[H*] System of differential equations usually have two or more unknown functions. For example, the Lotka-Volterra equations:

$$\begin{aligned}\frac{dx}{dt} &= ax - \alpha xy \\ \frac{dy}{dt} &= -cy + \gamma xy\end{aligned} \tag{4.7}$$

where $x(t)$ and $y(t)$ are the repsective populations of the prey and predator species. The constants a, α, c, and γ are based on empirical observations and depend on the particular species being studied.

[H*] The order of a differential equation is the order of the highest derivative that appears in the equation.

More genrally, the equation

$$F[t, u(t), u'(t), \dots, u^{(n)}(t)] = 0 \tag{4.8}$$

is an ordinary differential equation of the nth order.

Another one,

$$y''' + 2e^t y'' + yy' = t^3$$

is a third order differential equation for $y = u(t)$.

[H*] The ordinary differential equation

$$F(t, y, y', \dots, y^{(n)}) = 0$$

is said to be linear if F is a linear function of the variables $y, y', \dots, y^{(n)}$; a similar definition applies to partial differential equations.

Thus, the general linear ordinary differential equation of order n is

$$a_0(t)y^{(n)} + a_1(t)y^{(n-1)} + \cdots + a_n(t)y = g(t) \tag{4.9}$$

A simple physical problem that leads to a nonlinear differential equation is the oscillating pendulum. The angle θ that an oscillating pendulum of length L makes with the vertical direction satisfies the equation

$$\frac{d^2\theta}{dt^2} + \frac{g}{L}\sin\ \theta = 0 \tag{4.10}$$

the term involving $\sin\ \theta$ makes that equation to be nonlinear.

IV. First Order Ordinary Differential Equations

- Linear Equations; Method of Integrating Factors

[H*] Suppose we have this differential equations of first order

$$\frac{dy}{dt} = f(t, y)$$

where f is a given function of two variables. Any differentiable function $y = \phi(t)$ that satisfies this equation for all t in some interval is called a solution, and our object is to determine whether such functions exist.

For an arbitrary function f, there is no general method for solving the equation in terms of elementary functions.

[H*] We define the general first order linear equation in the standard form

$$\frac{dy}{dt} + p(t)y = g(t) \tag{4.11}$$

where p and g are given functions of the independent variable t.

To determine an appropriate integrating factor, we multiply Eq. (4.11) by an undetermined function $\mu(t)$, obtaining

$$\mu(t)\frac{dy}{dt} + p(t)\mu(t)y = \mu(t)g(t) \tag{4.12}$$

We see the left side of the Eq. (4.12) is the derivative of the product $\mu(t)y$, provided that $\mu(t)$ satisfies the equation

$$\frac{d\mu(t)}{dt} = p(t)\mu(t) \tag{4.13}$$

If we assume temporarily that $\mu(t)$ is positive, then we have

$$\frac{d\mu(t)/dt}{\mu(t)} = p(t)$$
$$\ln|\mu(t)| = \int p(t)\ dt + k$$

By choosing the arbitrary constant k to be zero, we obtain the simplest possible function for μ, namely,

$$\mu(t) = e^{\int p(t)\ dt} \tag{4.14}$$

Note that $\mu(t)$ is positive for all t, as we assumed. Returning to Eq. (4.12), we have

$$\frac{d}{dt}[\mu(t)y] = \mu(t)g(t) \tag{4.15}$$

Hence

$$\mu(t)y = \int \mu(t)g(t)\ dt + c \tag{4.16}$$

where c is an arbitrary constant.

The general solution of Eq. (4.11) is

$$y = \frac{1}{\mu(t)}\left[\int_{t_0}^{t} \mu(s)g(s)\ ds + c\right] \tag{4.17}$$

where t_0 is some convenient lower limit of integration.

[H*] **Example:**
Solve the initial value problem

$$ty' + 2y = 4t^2$$
$$y(1) = 2$$

Solution:
In order to determine $p(t)$ and $g(t)$ correctly, we must first rewrite the equation in the standard form of Eq. (4.11). Thus we have

$$y' + (2/t)y = 4t \tag{4.18}$$

so

$$p(t) = \frac{2}{t}$$
$$g(t) = 4t$$

Now, we first compute the integrating factor $\mu(t)$:

$$\begin{aligned}\mu(t) &= e^{\frac{2}{t}\ dt} \\ &= e^{2\ln|t|} \\ &= t^2\end{aligned}$$

Following the formula of Eq. (4.17), we obtain

$$\begin{aligned}y &= \frac{1}{\mu(t)}\left[\int_{t_0}^{t} \mu(s)g(s)\ ds + c\right] \\ &= \frac{1}{t^2}\left[\int_0^t s^2(4s)\ ds\right] \\ &= \frac{1}{t^2}\left[\int_0^t 4s^3\ ds\right] \\ &= \frac{1}{t^2}\left[t^4 + c\right] \\ y &= t^2 + \frac{c}{t^2}\end{aligned}$$

$y(t) = t^2 + \frac{c}{t^2}$ is the general solution of $ty' + 2y = 4t^2$.

To obtain the value for c, we will input the the initial condition value $y(1) = 2$ to the general solution

$$\begin{aligned}y(1) &= 2 \\ 2 &= 1^2 + \frac{c}{1^2} \\ c &= 1\end{aligned}$$

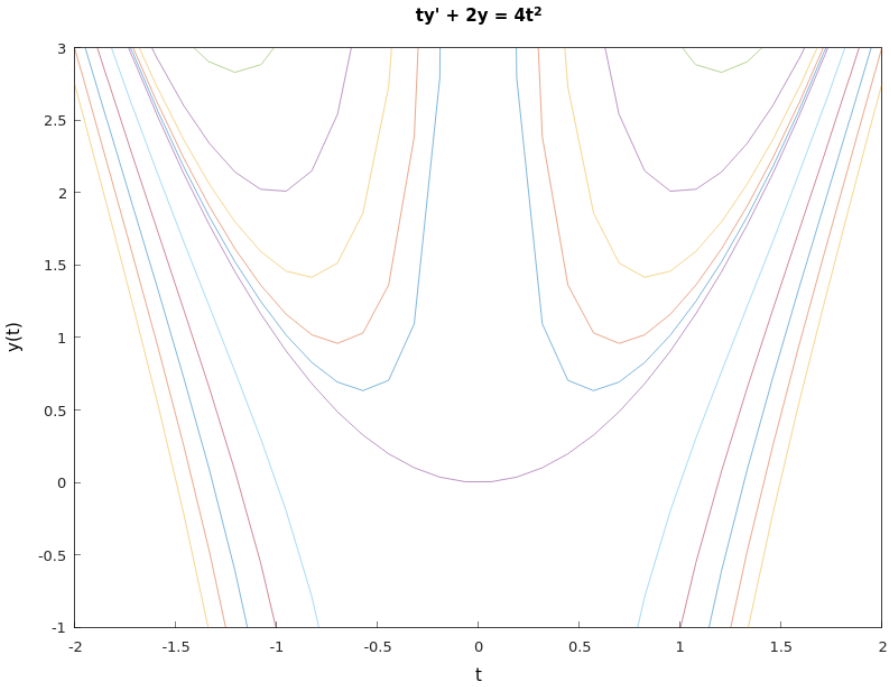

Figure 4.3: *The graphs shows the solutions for* $ty' + 2y = 4t^2$.

The solution becomes unbounded and is asymptotic to the positive y-axis as $t \to 0$ from the right. This is the effect of the infinite discontinuity in the coefficient $p(t)$ at the origin.

The function $y = t^2 + \frac{1}{t^2}$ for $t < 0$ is not part of the solution of this initial value problem. The solution fails to exist for some values of t. This is due to the infinite discontinuity in $p(t)$ at $t = 0$, which restricts the solution to the interval $0 < t < \infty$.

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;

int main() {
      using namespace hamzstplot;

      std::vector<double> t = linspace(—2 * pi, 2 * pi);
      std::vector<double> y1 = transform(t, [](auto t) { return
          pow(t,2) + 0.1/(pow(t,2)); });
      std::vector<double> y2 = transform(t, [](auto t) { return
          pow(t,2) + 0.23/(pow(t,2)); });
      std::vector<double> y3 = transform(t, [](auto t) { return
          pow(t,2) + 0.5/(pow(t,2)); });
      std::vector<double> y4 = transform(t, [](auto t) { return
          pow(t,2) + 1/(pow(t,2)); });
      std::vector<double> y5 = transform(t, [](auto t) { return
```

```
                pow(t,2) + 2/(pow(t,2)); });
            std::vector<double> y6 = transform(t, [](auto t) { return
                pow(t,2) + -1/(pow(t,2)); });
            std::vector<double> y7 = transform(t, [](auto t) { return
                pow(t,2) + -2/(pow(t,2)); });
            std::vector<double> y8 = transform(t, [](auto t) { return
                pow(t,2) + -3/(pow(t,2)); });
            std::vector<double> y9 = transform(t, [](auto t) { return
                pow(t,2) + -4/(pow(t,2)); });
            std::vector<double> y10 = transform(t, [](auto t) { return
                pow(t,2) + -5/(pow(t,2)); });
            std::vector<double> y11 = transform(t, [](auto t) { return
                pow(t,2); });

            plot(t, y1, t, y2, t, y3, t, y4, "-", t, y5,"-", t,y6, "-
                ", t,y7, "-", t,y8, "-", t,y9, "-", t,y10, "-", t,
                y11, "-");
            axis({-2, 2, -1, 3});
            title("ty' + 2y = 4t^2");
            xlabel("t");
            ylabel("y(t)");
            show();
            return 0;
    }
```

Code 15: *Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Solutions Curves for Method of Integrating Factors/main.cpp*

- Separable Equations

[H*] The general first order equation is

$$\frac{dy}{dx} = f(x,y) \tag{4.19}$$

By setting $M(x,y) = -f(x,y)$ and $N(x,y) = 1$ we will have

$$M(x,y) + N(x,y)\frac{dy}{dx} = 0 \tag{4.20}$$

If it happens that M is a function of x only and N is a function of y only, then it becomes

$$M(x) + N(y)\frac{dy}{dx} = 0 \tag{4.21}$$

Such an equation is said to be separable, because if it is written in the differential form

$$M(x)\,dx + N(y)\,dy = 0 \tag{4.22}$$

The differential form (4.22) is also more symmetric and tends to suppress the distinction between independent and dependent variables.

A separable equation can be solved by integrating the functions M and N.

[H*] **Example:**
Show that the equation

$$\frac{dy}{dx} = \frac{x^2}{1-y^2}$$

is separable, and then find an equation for its integral curves.

Solution:

If we write the equation as

$$-x^2 + (1-y^2)\frac{dy}{dx} = 0$$

then it is separable. Recall from calculus that if y is a function of x, then by the chain rule

$$\frac{d}{dx}f(y) = \frac{d}{dy}f(y)\frac{dy}{dx} = f'(y)\frac{dy}{dx}$$

If $f(y) = y - \frac{y^3}{3}$, then

$$\frac{d}{dx}\left(y - \frac{y^3}{3}\right) = (1-y^2)\frac{dy}{dx}$$

Thus,

$$\begin{aligned} -x^2 + (1-y^2)\frac{dy}{dx} &= 0 \\ (1-y^2)dy &= x^2\,dx \\ y - \frac{y^3}{3} &= \frac{x^3}{3} + c \\ -x^3 + 3y - y^3 &= c \end{aligned}$$

Any differentiable function $y = \phi(x)$ that satisfies $-x^3 + 3y - y^3 = c$ is a solution of $\frac{dy}{dx} = \frac{x^2}{1-y^2}$.

An equation of the integral curve passing through a particular point (x_0, y_0) can be found by substituting x_0 and y_0 for x and y, respectively, in $-x^3 + 3y - y^3 = c$ and determining the corresponding value of c.

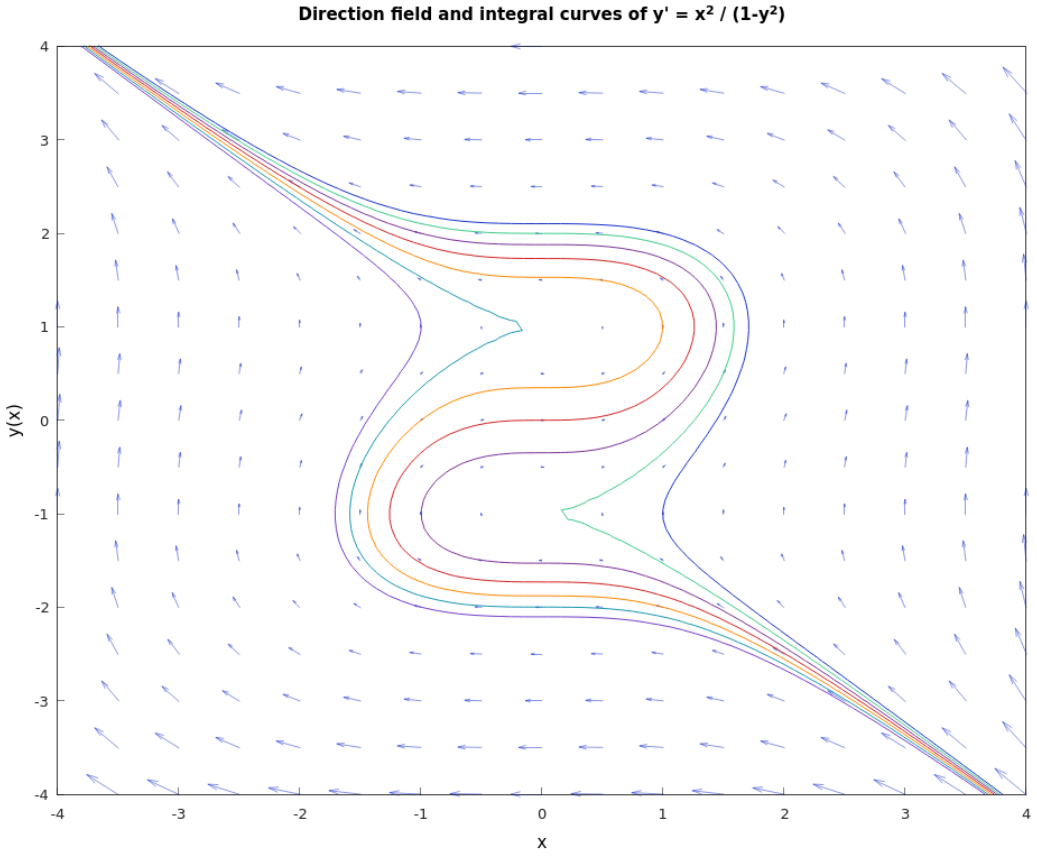

Figure 4.4: *The graphs shows the direction field and integral curves of* $y' = \frac{x^2}{1-y^2}$.

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;

int main() {
	using namespace hamzstplot;

	vector_2d newcolors = {{0.83, 0.14, 0.14},
		{1.00, 0.54, 0.00},
		{0.10, 0.6, 0.70},
		{0.47, 0.25, 0.80},
		{0.50, 0.2, 0.60},
		{0.25, 0.80, 0.54}};
	colororder(newcolors);

	auto [x, y] = meshgrid(iota(-4, 0.5, 5), iota(-4, 0.5, 4))
	    ;
	vector_2d u = transform(x, y, [](double x, double y) {
```

```
            return 1-pow(y,2) ; }); // dx
        vector_2d v = transform(x, y, [](double x, double y) {
            return pow(x,2) ; }); // dy

        quiver(x, y, u, v);
        hold(on);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) ; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) - 1; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) - 2; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) - 3; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) + 1; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) + 2; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) + 3; },"-")->line_width(1);
        hold(off);

        axis({-4, 4, -4, 4});
        title("Integral curves of y' = x^{2} / (1-y^{2})");
        xlabel("x");
        ylabel("y(x)");
        show();
        return 0;
    }
```

Code 16: *Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Direction Field and Solutions Curves for Separable Equations/main.cpp*

The coding here is using **fimplicit** since we are inputting $-x^3 + 3y - y^3 - c = 0$ as the function to be drawn, it is the implicit function. We also draw the direction field so we are using **hold(on)** then closing it with **hold(off)**.

- Modeling with First Order Equations

[H*] Differential equations are of interest to nonmathematicians primarily because of the possibility of using them to investigate a wide variety of problems in the physical, biological, and social sciences.

One reason for this is that mathematical models and their solutions lead to equations relating the variables and parameters in the problem. These equations often enable you to make predictions about how the natural process will behave in various circumstances.

Mathematical modeling and experiment or observation are both critically important

and have somewhat complementary roles in scientific investigations. Mathematical models are validated by comparison of their predictions with experimental results.

[H*] Three identifiable steps that are always present in the process of mathematical modeling:

1. Construction of the Model.
2. Analysis of the Model.
3. Comparison with Experiment or Observation.

[H*] **Compound Interest**

Suppose that a sum of money is deposited in a bank that pays interest at an annual rate r. The value $S(t)$ of the investment at any time t depends on the frequency with which interest is compounded as well as on the interest rate.

Financial institutions have various policies concerning compounding: some compound monthly, some weekly, some even daily.

If we assume that compounding takes place continuously, then we can set up a simple initial value problem that describes the growth of the investment.

The rate of change of the value of the investment is $\frac{dS}{dt}$, and this quantity is equal to the rate at which interest accrues, which is the interest rate r times the current value of the investment $S(t)$. Thus

$$\frac{dS}{dt} = rS \tag{4.23}$$

is the differential equation that governs the process. Suppose that we also know the value of the investment at some particular time, say,

$$S(0) = S_0 \tag{4.24}$$

Then the solution of the initial value problem at Eq. (4.23) and Eq. (4.24) gives the balance $S(t)$ in the account at any time t. This initial value problem is readily solved, since the differential equation at Eq. (4.23) is both linear and separable. Consequently, by solving Eqs. (4.23) and (4.24), we find that

$$S(t) = S_0 e^{rt} \tag{4.25}$$

Thus a bank account with continuously compounding interest grows exponentially.

Let us suppose that there may be deposits or withdrawals in addition to the accrual of interest, dividends, or capital gains. If we assume that the deposits or withdrawals take place at a constant rate k, then Eq. (4.23) is replaced by

$$\frac{dS}{dt} = rS + k \tag{4.26}$$

where k is positive for deposits and negative for withdrawals.

Eq. (4.26) is linear with the integrating factor e^{-rt}, so its general solution is

$$S(t) = ce^{rt} - \frac{k}{r} \tag{4.27}$$

where c is an arbitrary constant. To satisfy the initial condition Eq. (4.24), we must choose $c = S_0 + \frac{k}{r}$. Thus the solution of the initial value problem is

$$S(t) = S_0 e^{rt} + \left(\frac{k}{r}\right)(e^{rt} - 1) \tag{4.28}$$

The first term in the expression (4.28) is the part of $S(t)$ that is due to the return accumulated on the initial amount S_0, and the second term is the part that is due to the deposit or withdrawal rate k.

The advantage of stating the problem in this general way without specific values for S_0, r, or k lies in the generality of the resulting formula (4.28) for $S(t)$. With this formula we can readily compare the results of different investment programs or different rates of return.

[H*] **Example:**
Suppose that one person opens an individual retirement account (IRA) at age 25 and makes annual investments of USD 2000 thereafter in a continuous manner. Assuming a rate of return of 8%, what will be the balance in the IRA at age 65?

Solution:

We have

$$\begin{aligned} S_0 &= 0 \\ r &= 0.08 \\ k &= 2000 \end{aligned}$$

and we wish to determine $S(40)$. From Eq. (4.28) we have

$$\begin{aligned} S(40) &= (0)e^{(0.08)(40)} + \left(\frac{2000}{0.08}\right)(e^{(0.08)(40)} - 1) \\ &= (25,000)(e^{3.2} - 1) \\ &= 588,313 \end{aligned}$$

It is interesting to note that the total amount invested is USD 80,000 ($40 \times 2,000$), so the remaining amount of USD 508,313 results from the accumulated return on the investment. The balance after 40 years is also fairly sensitive to the assumed rate.

For the C++ code, we are using Hamzstlab Mathematics for the first time as the introduction, this is to complement Hamzstplot beautifully. Hamzstlab Mathematics is using ImGUI and Implot so it can make the interactive GUI like in this example.

We can of course plot $S(t)$ easily with various rate with Hamzstplot, but we want a slider that can change the plot at an instant, that is why we are using Hamzstlab Mathematics, it can do the job better and faster.

We create a few examples for the first order ordinary differential equations section for Hamzstlab Mathematics. The code to create this is longer, not only one CPP file, but more than one, and it is a lot different than Hamzstplot, a lot of learning, you have to

read a lot for the codes, read about ImGUI, Implot, steeper curve of learning, but it will worth your time since it can sustain not only decades but till another 100 years for the durability.

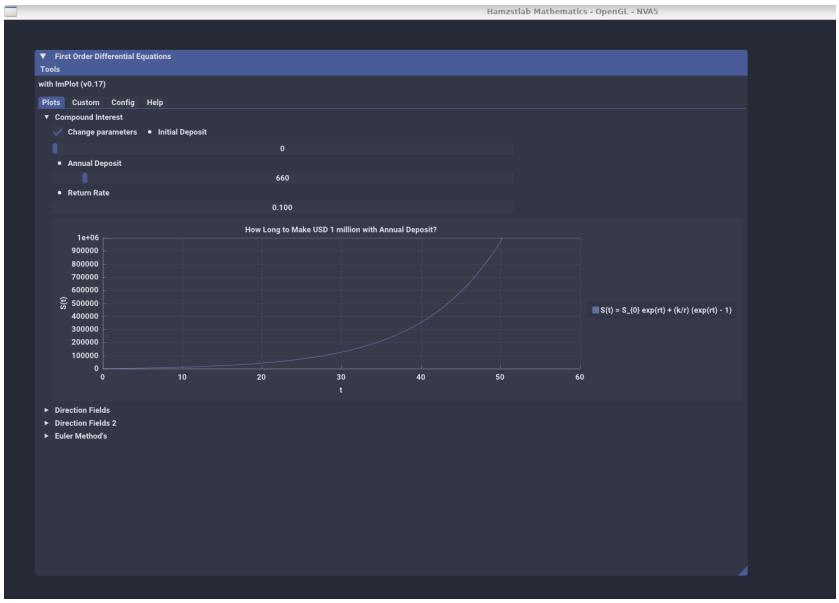

Figure 4.5: *The interactive plot of $S(t)$ with Hamzstlab-Mathematics you can change the parameters and the plot will adjust with the parameters that you just input.*

```
...
...

void Demo_CompoundInterest() {
static float xs1[1001], ys1[1001];
static int S0 = 0;
static float r = 0.08;
static int k = 2000;
for (int i = 0; i < 1001; ++i) {
xs1[i] = i;
ys1[i] = S0 * exp(r*i) + (k/r)*(exp(r*i) — 1) ;
}

static bool range = false;
ImGui::Checkbox("Change parameters", &range);

if (range) {
ImGui::SameLine();
ImGui::SetNextItemWidth(200);
ImGui::BulletText("Initial Deposit");
ImGui::SliderInt("##Initial Deposit", &S0, 0, 10000);
ImGui::SetNextItemWidth(200);
```

```
        ImGui::BulletText("Annual Deposit");
        ImGui::SliderInt("##Annual Deposit", &k, 1, 10000);
        ImGui::SetNextItemWidth(200);
        ImGui::BulletText("Return Rate");
        ImGui::DragFloat("##Return rate", &r, 0.01f, 0.2f);
        ImGui::SetNextItemWidth(200);
        }
        if (ImPlot::BeginPlot("How Long to Make USD 1 million with Annual
            Deposit?")) {
        ImPlot::SetupAxes("t","S(t)");
        ImPlot::SetupAxesLimits(0, 60, 0, 1000000);
        ImPlot::SetupLegend(ImPlotLocation_East, ImPlotLegendFlags_Outside
            );

        ImPlot::PlotLine("S(t) = S_{0} exp(rt) + (k/r) (exp(rt) — 1)",
            xs1, ys1, 1001);
        ImPlot::EndPlot();
        }
        }
        ...
        ...
```

Code 17: *Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp*

We only include the snippet of code to plot the $S(t)$, the full code can be seen at Hamzstlab Mathematics from the repository of Hamzstlab Mathematics at the **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**, to run the example with Makefile you can go to this directory **Hamzstlab-Mathematics/examples/First Order Differential Equation**.

- Differences Between Linear and Nonlinear Equations

  [H*]

- Autonomous Equations and Population Dynamics

  [H*]

- Exact Equations and Integrating Factors

  [H*]

- Numerical Approximations: Euler's Method

  [H*]

V. Higher Order Linear Equations

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

VI. Boundary Value Problems and Sturm-Liouville Theory

- The Occurrence of Two-Point Boundary Value Problems

  [H*] The first
  [H*]

- Nonhomogeneous Boundary Value Problems

  [H*] The first
  [H*]

Chapter 5

Probability and Statistics

$'' - x$

D

Chapter 6

Partial Differential Equations

″″ - x

D·

Bibliography

[1] Boyce, William E., DiPrima, Richard C. (2010) Elementary Differential Equations and Boundary Value Problems 9th Edition, John Wiley & Sons, Hoboken, New Jersey, United States.

[2] Colley, Susan Jane (2012) Vector Calculus 4th Edition, Pearson, Boston, MA, United States.

[3] Dorf, Richard C., Bishop, Robert H. (2010) Modern Control Systems 12th Edition, Pearson, New Jersey, United States.

[4] Gezerlis, Alex (2020) Numerical Methods in Physics with Python, Cambridge University Press, Cambridge, United Kingdom.

[5] Purcell, Rigdon, Varberg (2007) Calculus 9th Edition, Pearson, Upper Saddle River, New Jersey, United States.

[6] Kenneth H. Rosen (2006) Discrete Mathematics and its Applications 6th Edition, McGraw-Hill, Boston, United States.

[7] Anton, Rorres (2006) Elementary Linear Algebra with Supplemental Applications 10th Edition, Wiley, Boston, United States.

[8] Anton, Howard, Rorres, Chris, Elementary Linear Algebra with Supplemental Applications 12th edition, Wiley, Hoboken, New Jersey, United States.